# Comparing the Impact and Arrival Cost of Naive and OBI-Enhanced Order Splitting Strategies

**Michael Cusack-Nelkin** mc5882@columbia.edu
**Alick Zhou** az2960@columbia.edu
**Debby Zhao** lz3083@columbia.edu
**Michael Bakshandeh** mb5499@columbia.edu
**Vijeet Prasad** vp2580@columbia.edu

## Abstract

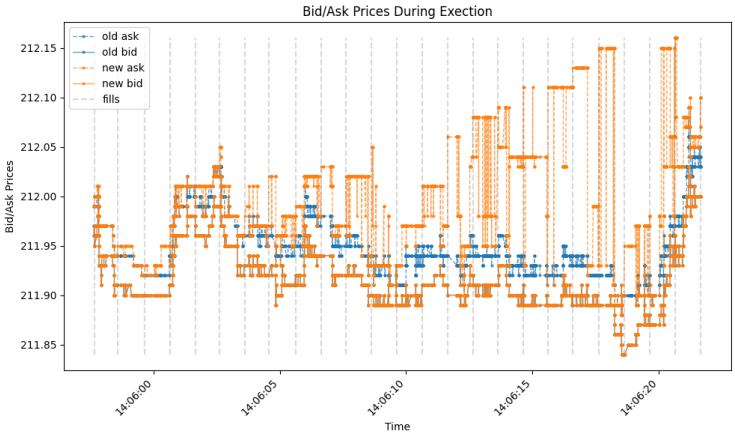
problem (naive fills = bad) -> goals (use OBI to improve things) -> methods (obi informs aggression) -> results -> significance (?) Definitions of key terms and calculations is provided in Section 5.

## 1 Motivation and Main Ideas

A fundamental hurdle in medium-frequency systematic trading is the balance between arrival cost and immediacy. Submitting a very large marketable order leads to significant price impact as it takes liquidity from the limit order book. On the other hand, splitting a parent order into smaller chunks over seconds or minutes exposes child fills to adverse price drifts, and when done naively also leads to significant price impact. The goal of this work is to reduce the total arrival cost of a parent order by intelligently calibrating the aggression of child orders to take advantage of favorable price movements while avoiding unfavorable ones.

### 1.1 Underperformance of Naive Splitting

Unfortunately, naively splitting a parent order into $n$ chunks and filling them as market orders $\tau$ time units apart has high impact and exposes the parent order to significant price drifts. The below example of splitting a 5000-share order into twenty-five 200-share buy fills shows the significant impact of repeatedly lifting the ask overtime. The underperformance of this strategy nessesitates a more intelligent way to split and fill large orders.




### 1.2 Book Imbalance as a Primary Signal

We chose to use the imbalance in quantity on the top six levels of the book as our signal to calibrate the aggression of our fill strategy. We made this choice because OBI is a well understood, easy to

implement, and powerful signal of price direction on the micro-milli second time scale. To explore OBI's predictive power, we used the markout plot below. Within five separate ranges of OBI values, we sample 100,000 timestamps and calculate the difference between the initial mid-price and the mid-price at 10,000 subsequent times, then average over each of the 100,000 paths. We use a time horizon of $[0, 1]$ second, in tens of microseconds.

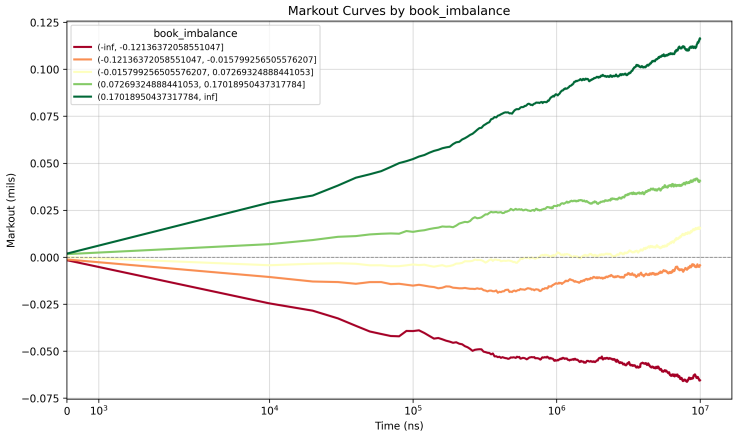



Figure 1: Markout Plot

This plot clearly shows that OBI is a powerful, structural predictor of price direction within a few milliseconds and that this price motion is structural, indicated by the length of time that the markouts stay smooth and near-zero variance. Using this predictive power, our fill strategy can reliably predict positive and negative price drifts and take advantage of them.

## 2 Methodological Choices and Innovations

### 2.1 Sourcing Data

We utilize the `MBP-10` schema from Databento, which provides the top 10 levels of the book for a selection of stocks and date range. The dataset includes all order book updates (trades, adds, cancels, etc.) which allows us to reconstruct the book at every book update. The data is sourced from Databento. We fetch data through a simple `DatabentoGenericAdapter` class that uses a hierarchical caching structure. This allows us to call the Databento API only for new information, which is necessary to manage storage and memory for L2 book data. The adapter caches on `[tickers, start, end, dataset, schema]`. Here, our schema is always `MBP-10` and we use the `XNAS.ITCH` Dataset which gives us full access to the Nasdaq exchange.

We use L2 data for three tickers, `AMZN`, `CSCO`, and `DRH` from `"2026-03-18":"2026-04-18"`. We chose these tickers because they cover different varieties of equities. `AMZN` is a liquid small-tick stock that should offer ample resilience to price impact and relatively high and volatile bid-ask spreads. `CSCO` is a liquid large-tick stock that still has deep liquidity but larger spreads and more variability throughout the day. Finally, `DRH` is an illiquid large-tick stock that offers a challenge to minimizing costs. It cannot accept large amounts of liquidity and moves more slowly than `AMZN` and `CSCO`. Put together, this dataset is 28.7GB, on the order of RAM capacity for a sequential process but requires some management for parallel processing tasks, like the below.

### 2.2 Simulator

### 2.3 Strategy Evaluation

To evaluate each of the following strategies, we utilize a parallel processing trial workflow. At initialization, we set the numerous required hyperparameters, each listed below.

- `Latency = 100us`, `Poisson Replenishment = 15.0`, `PARENT_QTY = 10_000`, `MAX_WORKERS = 10`, `N_TRIALS = 5_000`, `SEED = 12345`, `HORIZON = 300s`

Then we scan the full dataset's parquet cache and split into one `.parquet` for each ticker. We then randomly sample `N_TRIALS = 5_000` initial timestamps from the L2 dataset, ensuring that no

two windows from the initial timestamp plus `HORIZON = 300s` seconds overlap more than a few seconds. Then for each combination of ticker and fill strategy, we initialize `MAX_WORKERS = 10` CPU workers and scan the relavent ticker `.parquet` file for each window and run the fill strategy on that window only, randomly choosing the side of the order from `['BUY', 'SELL']`. This workflow balances memory requirements and parallelized processing speed, which brought down the time cost of running trials half an hour per trial sequentially over the full dataset to 5000 trials in a few minutes (depending on the ticker).

To measure the performance of each fill strategy, we save information including `side`, `parent_filled`, `parent_arrival_cost`, `mean_child_arrival_cost`, `median_child_arrival_cost`, and `parent_lifetime`. Below we compare our final successful strategy with the naive strategy using the above metrics, and go over some failed strategies and their associated learnings along the way. More on specific definitions of the above metrics can be found in Section 5.

### 2.4 Naive Benchmark

As discussed and plotted earlier, we are comparing our stragies with the naive method of splitting orders into `N_CHUNKS = 50`, and filling them a fixed time interval `WAIT_TIME = 1s` apart using regular market orders. Below in Table 1 are some performance metrics for this naive method to serve as a benchmark we are competing against.

Table 1: Naive Results

| statistic<br>str | total_filled<br>f64 | parent_cost<br>f64 | mean_child_cost<br>f64 | median_child_arrival_cost<br>f64 | lifetime_s<br>f64 |
|---|---|---|---|---|---|
| "mean" | 10000.0 | 9.8626 | 5.798219 | 4.182775 | 9.582253 |
| "min" | 10000.0 | -21.535187 | 1.374581 | 0.580619 | 6.221948 |
| "25%" | 10000.0 | 6.428417 | 3.883612 | 3.316627 | 9.524487 |
| "50%" | 10000.0 | 9.053304 | 5.149228 | 3.90504 | 9.687721 |
| "75%" | 10000.0 | 12.472484 | 7.069212 | 4.637834 | 9.761385 |
| "max" | 10000.0 | 61.71924 | 36.425326 | 35.814917 | 9.800094 |

### 2.5 Failure: Naive with IOC

The first pass attempt to improve on the benchmark is simply running the same strategy using marketable IOC limit orders instead of market orders. Results below in Table 2. We used the `DEPTH_LMT = "09"` parameter to set the limit price of each child order to the 10th level of the ask (bid) for a parent buy (sell) order at the initial time of execution. This resulted in better fills, however it almost never completed hte full parent order given it's simple cap on the allowed execution price.

Table 2: Naive IOC Results

| statistic<br>str | total_filled<br>f64 | parent_cost<br>f64 | mean_child_cost<br>f64 | median_child_arrival_cost<br>f64 | lifetime_s<br>f64 |
|---|---|---|---|---|---|
| "mean" | 7818.5352 | 5.28774 | 3.374856 | 3.293796 | 8.880517 |
| "min" | 200.0 | -21.525161 | 1.207293 | 0.580619 | 0.0 |
| "25%" | 6838.0 | 4.482656 | 3.052005 | 2.950195 | 8.446878 |
| "50%" | 7905.0 | 5.518874 | 3.353437 | 3.200819 | 9.370963 |
| "75%" | 8967.0 | 6.410203 | 3.658963 | 3.726398 | 9.709241 |
| "max" | 10000.0 | 29.237672 | 16.116962 | 15.40023 | 9.800094 |

### 2.6 Failure: OBI Aggressive Only

Beyond simply tweaking the benchmark our first true pass at an improvement used OBI to time our fills using the parameter `OBI_THRESHOLD = 0.85`. For a buy order, if the OBI was above the threshold that indicated an impending upward price move, so the logic was to fill a buy order immediately of size '$MAX_SIZE' * |OBI|$, wehre `MAX_SIZE = 350`. This would dynamically size orders based on confidence about subsequent price moves, and attempt to only fill at advantageous times. Below is a plot of this strategy in action.

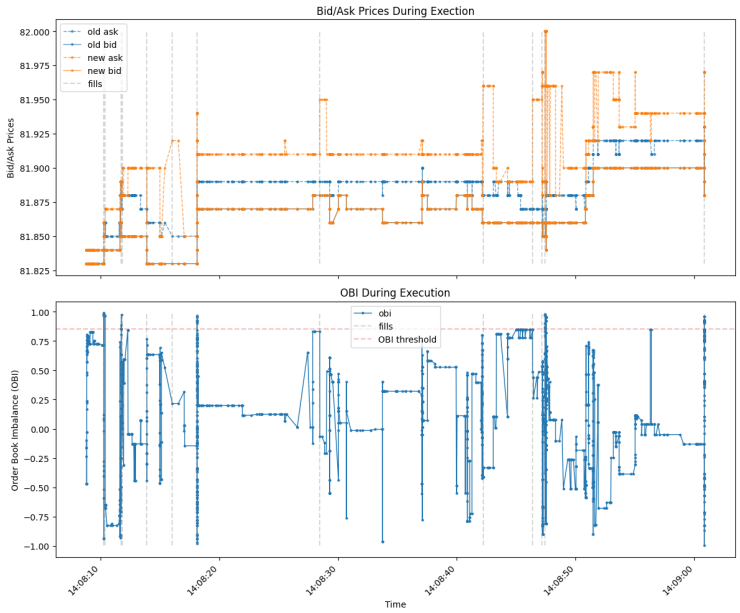



Figure 2: OBI Aggressive Only Fills

This strategy had a few issues. Firstly, calibrating the parameters above was difficult, and for some time windows, the OBI simply never hit the threshold it never filled. More crucially though, this method led us to the final improvement and a successful strategy. This strategy originated from an *alpha-generation perspective*. Using OBI to *time* orders is what a strategy would to in order to *enter positions* and take advantage of future returns. It took some time and effort to escape this way of thinking and reach an *execution* based perspective, where a strategy's goal is to *minimize cost* by setting up a sequence of *fills* (not changing positions) over a set time horizon. Allowing the price to drift toward and away from the ideal fill and taking advantage of favorable conditions, not predicting returns to be captured.

# 3 Empirical Findings

## 3.1 Successful Strategy: Hybrid OBI Calibrated Aggression

## 3.2 Test Stats / Significance

Table 4: T-Test Significance

| symbol<br>str | hybrid mean<br>f64 | naive mean<br>f64 | t_stat<br>f64 | p_val<br>f64 |
|---|---|---|---|---|
| "AMZN" | 0.18 | 1.59 | 35.97 | 0.0 |
| "CSCO" | 1.5 | 5.24 | 56.5 | 0.0 |
| "DRH" | 25.47 | 54.43 | 48.42 | 0.0 |

## 3.3 Effects of Latency

Table from ilestone showing the effect of latency on hybrid filler

Table 5: Effects of Latency

| symbol<br>str | hybrid mean<br>f64 | naive mean<br>f64 | latency (ns)<br>f64 | latency_level<br>str |
|---|---|---|---|---|
| "CSCO" | 4.19 | 9.76 | 10000.0 | "low" |
| "AMZN" | 1.36 | 4.3 | 10000.0 | "low" |

Table 5: Effects of Latency

| symbol<br>str | hybrid mean<br>f64 | naive mean<br>f64 | latency (ns)<br>f64 | latency_level<br>str |
|---|---|---|---|---|
| "DRH" | 35.53 | 88.32 | 10000.0 | "low" |
| "CSCO" | 4.19 | 9.86 | 100000.0 | "mid" |
| "AMZN" | 1.42 | 4.17 | 100000.0 | "mid" |
| "DRH" | 35.62 | 88.36 | 100000.0 | "mid" |
| "CSCO" | 4.38 | 10.27 | 5e7 | "high" |
| "AMZN" | 1.35 | 4.37 | 5e7 | "high" |
| "DRH" | 35.64 | 88.54 | 5e7 | "high" |

### 3.4 Effects of Replenishment

Table 6: Effects of Replenishment

| symbol<br>str | hybrid mean<br>f64 | naive mean<br>f64 | replenishment<br>f64 | replenishment_level<br>str |
|---|---|---|---|---|
| "CSCO" | 5.37 | 13.63 | 1.0 | "low" |
| "AMZN" | 2.03 | 6.9 | 1.0 | "low" |
| "DRH" | 42.01 | 94.8 | 1.0 | "low" |
| "CSCO" | 4.19 | 9.86 | 15.0 | "mid" |
| "AMZN" | 1.42 | 4.17 | 15.0 | "mid" |
| "DRH" | 35.62 | 88.36 | 15.0 | "mid" |
| "CSCO" | 3.89 | 8.03 | 29.0 | "high" |
| "AMZN" | 1.35 | 3.09 | 29.0 | "high" |
| "DRH" | 35.05 | 84.7 | 29.0 | "high" |

### 3.5 Intraday Seasonality

## 4 Future Work

### 4.1 Smoothed OBI

Tried it, got some weird results (no hyperparam tuning, possibly a quirk of replenishment.)

### 4.2 A Catch-up/Fall-Behind Mechanism

Currently waiting in-band.

### 4.3 Fixed horizon vs T(v)

## 5 Definitions and Appendix

Throughout this work we refer to a few concepts that require specific definitions. We lay them out below for reference.

- OBI: We define order book imbalnce (OBI) using the top *six* levels of the book, as follows:

$$OBI = \frac{\Sigma_{i=0}^{5} Q_i^{bid} - \Sigma_{i=0}^{5} Q_i^{ask}}{\Sigma_{i=0}^{5} [Q_i^{bid} + Q_i^{ask}]} \in [-1, 1] \tag{1}$$

- Price Impact : When referring to price impact we inlcude only the mechanical impact of taking liquidity from the limit order book. Below is a visualization of the mechanical impact for reference. Our order book simulation is discussed in Section 2.2, and includes further information on impact as well as replenishment adn latency.

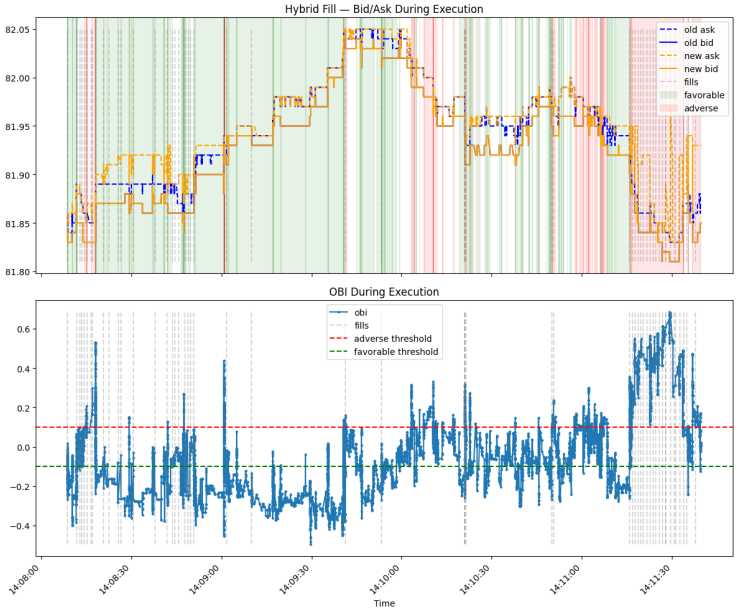



Figure 3: Hybrid Plot

Table 3: OBI Calibrated Aggression

(a) AMZN

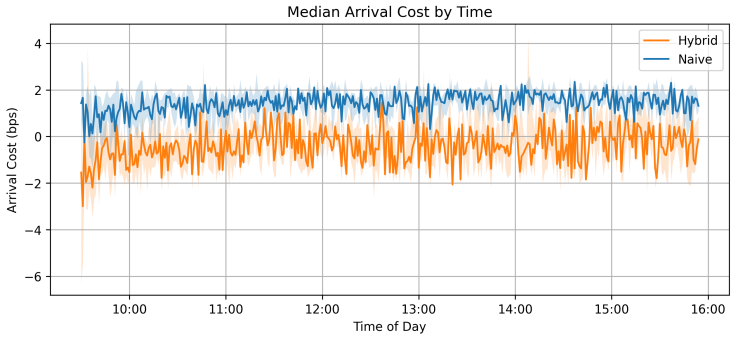
| statistic | parent_cost | mean_child_cost | lifetime_s |
|---|---|---|---|
| str | f64 | f64 | f64 |
| "mean" | 0.184687 | -0.508529 | 28.482708 |
| "min" | -18.378149 | -9.542207 | 0.0 |
| "25%" | -1.215225 | -1.549884 | 6.433403 |
| "50%" | 0.069449 | -0.757397 | 13.0227 |
| "75%" | 1.365203 | 0.507965 | 27.275029 |
| "max" | 40.170454 | 2.330946 | 299.604724 |

(a) CSCO

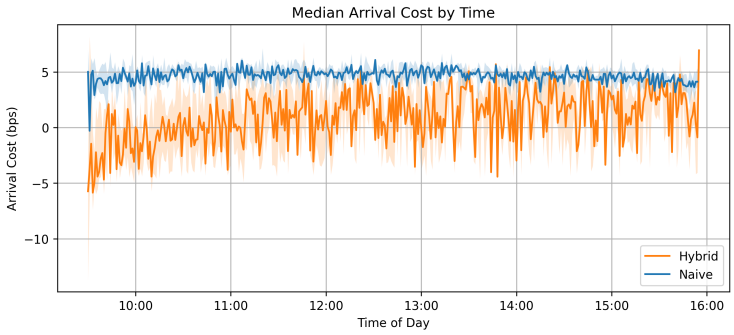
| statistic | parent_cost | mean_child_cost | lifetime_s |
|---|---|---|---|
| str | f64 | f64 | f64 |
| "mean" | 1.496204 | -0.109473 | 32.403243 |
| "min" | -37.34349 | -13.824266 | 0.0 |
| "25%" | -2.495476 | -3.533627 | 9.64752 |
| "50%" | 2.133194 | 0.249334 | 15.597951 |
| "75%" | 5.022137 | 3.028982 | 28.05321 |
| "max" | 27.74481 | 5.518933 | 299.878286 |

(a) DRH

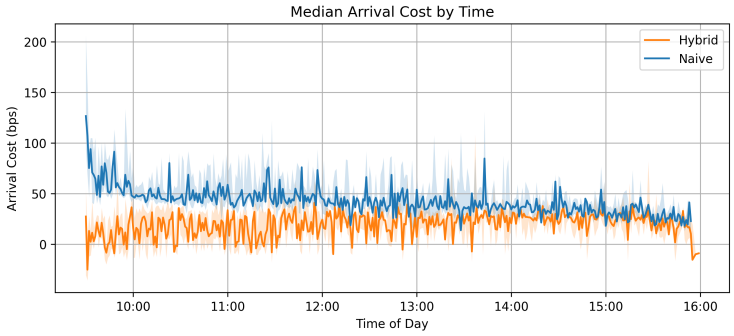
| statistic | parent_cost | mean_child_cost | lifetime_s |
|---|---|---|---|
| str | f64 | f64 | f64 |
| "mean" | 25.47164 | 10.770199 | 75.61788 |
| "min" | -62.935586 | -92.823939 | 0.0 |
| "25%" | 0.963938 | -5.316321 | 9.051737 |
| "50%" | 28.342588 | 15.368545 | 31.240362 |
| "75%" | 40.402367 | 23.090175 | 115.460249 |
| "max" | 390.002103 | 200.59842 | 299.998009 |




(a) AMZN

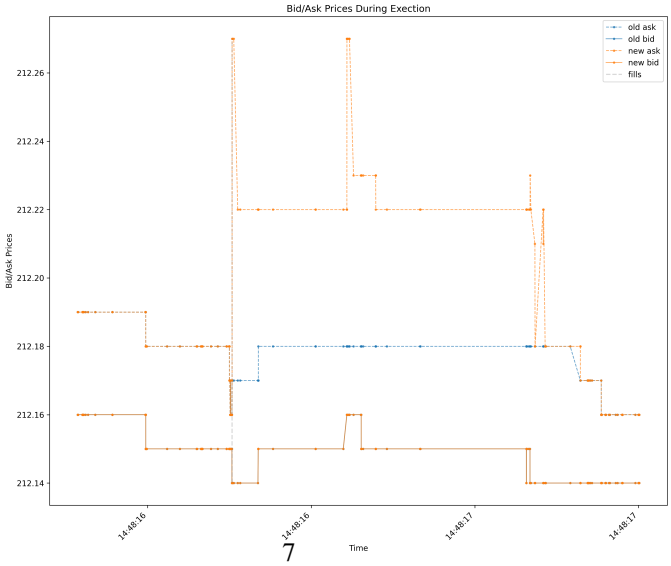



(b) CSCO




(c) DRH

Figure 7: Intraday Seasonality of Arrival Cost



- Arrival Cost: When referring to arrival cost of a parent or child order, we are referring to the volume-weighted average price of a fill minus the mid price at the time of execution, divided by mid price and converted to bps. For a parent order, the VWAP sums over child order's individual vwaps in the same way, and the mid price is set to the price at the time the parent order sent to the execution engine. For child orders, the mean and median arrival costs are calculated across child orders for a given parent. Below, $q_i$ and $p_i$ are the individual fill's quantities and prices respectively.

$$\begin{aligned} p_{vwap} &= \frac{\Sigma_{i=0}^{n} q_i \, p_i}{\Sigma_{i=0}^{n} q_i} \\ \text{ArrivalCost (bps)} &= \frac{p_{vwap} - p_{mid}}{p_{mid}} * 10^4 \end{aligned} \tag{2}$$